PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Graduação em Engenharia de Computação - Disciplina de Sistemas Reconfiguráveis

RODRIGO PRETES MAIA
ISAAC ALVES SCHUENCK

DESCRIÇÃO DE CIRCUITOS LÓGICOS SEQUENCIAIS UTILIZANDO VHDL

Belo Horizonte
12 de novembro de 2023

RODRIGO PRETES MAIA
ISAAC ALVES SCHUENCK

DESCRIÇÃO DE CIRCUITOS LÓGICOS SEQUENCIAIS UTILIZANDO VHDL

Trabalho desenvolvido como parte dos projetos
da disciplina de Sistemas Reconfiguráveis

Professor: Antônio Hamilton Magalhaes

Belo Horizonte 2023

Lista de Figuras

Figura 1: Diagrama equivalente do W_REG.
Figura 2: Bibliotecas e Entidade W_reg.
Figura 3: Arquitetura W_REG.
Figura 4: Funcionamento W_REG.
Figura 5: Diagrama equivalente do FSR_REG.
Figura 6: Entidade FSR_REG.
Figura 7: Arquitetura FSR_REG.
Figura 8: Funcionamento FSR_REG.
Figura 9: Diagrama equivalente do STATUS_REG.
Figura 10: Bibliotecas e Entidade STATUS_REG.
Figura 11: Arquitetura STATUS_REG.
Figura 12: Saídas arquitetura STATUS_REG.
Figura 13: Funcionamento 1 STATUS_REG.
Figura 14: Funcionamento 2 STATUS_REG.
Figura 15: Diagrama equivalente do STACK.
Figura 16: Bibliotecas e Entidade STACK.
Figura 17: Arquitetura Stack.
Figura 18: Funcionamento STACK.
Figura 19: Diagrama equivalente do PC_REG.
Figura 20: Bibliotecas e Entidade PC_REG.
Figura 21: Arquitetura PC_REG.
Figura 22: Arquitetura 2 PC_REG.
Figura 23: Funcionamento 1 PC_REG.
Figura 24: Funcionamento 2 PC_REG.
Figura 25: Funcionamento 3 PC_REG.

Sumário

I. INTRODUÇÃO 5
II. WR_REG 5
II.I - Entradas e Saídas 5
II.II - Diagrama equivalente 5
II.III - Descrição W_REG em VHDL 5
II.IV - Simulação e teste no Quartus II 6
III. FSR_REG 6
III.I - Entradas e Saídas 7
III.II - Diagrama equivalente 7
III.III - Descrição FSR_REG em VHDL 7
III.IV - Simulação e teste no Quartus II 8
IV. STATUS_REG 9
IV.I - Entradas e Saídas 9
IV.II - Diagrama equivalente 10
IV.III - Descrição STATUS_REG em VHDL 11
IV.IV - Simulação e teste no Quartus II 11
V. STACK 12
V.I - Entradas e Saídas 13
V.II - Diagrama equivalente 14
V.III - Descrição STACK em VHDL 14
V.IV - Simulação e teste no Quartus II 15
VI. PC_REG 15
VI.I - Entradas e Saídas 15
VI.II - Diagrama equivalente 16
VI.III - Descrição PC_REG em VHDL 16
VI.IV - Simulação e teste no Quartus II 18
VII. CONCLUSÃO 19
VIII. REFERENCIAS 19

I. INTRODUÇÃO

Neste trabalho, abordaremos a criação, implementação e simulação de testes dos registradores propostos para esta etapa do projeto, que incluem o W_REG, FSR_REG, STATUS_REG, STACK e PC_REG. Utilizaremos a linguagem de descrição VHDL no software Quartus II. Inicialmente, detalharemos o funcionamento do W_REG no primeiro bloco, em seguida, apresentaremos as informações relacionadas à FSR_REG no segundo bloco, e assim sucessivamente.

II. WR_REG

O registrador W_REG é composto por um registrador de 8 bits, contendo 4 entradas e uma saída de 8 bits. Ele inclui uma entrada nRST, ativada em nível lógico baixo, cuja ativação resulta na reinicialização de todos os bits da saída, sendo priorizada em relação a todas as outras entradas. Além disso, o W_REG possui uma entrada CLK_IN, que é um sinal de clock utilizado para habilitar a operação de escrita no registrador. A terceira entrada é D_IN, a qual possui 8 bits e representa o valor a ser gravado no registrador, desde que a entrada WR_EN esteja ativa. A entrada WR_EN é responsável por habilitar a operação de escrita no registrador na borda de subida do sinal de clock.

II.I - Entradas e Saídas

Entradas:

- nRST – Entrada de reset assíncrono, ativada em nível lógico baixo.
- CLK_IN – Entrada de clock para escrita no registrador.
- D_IN [7...0] – Entrada de dados para escrita no registrador, com habilitação através de WR_EN.
- WR_EN – Entrada para endereçamento indireto.
-

Saídas:

- W_OUT [7...0] - Saída de dados.

II.II - Diagrama equivalente

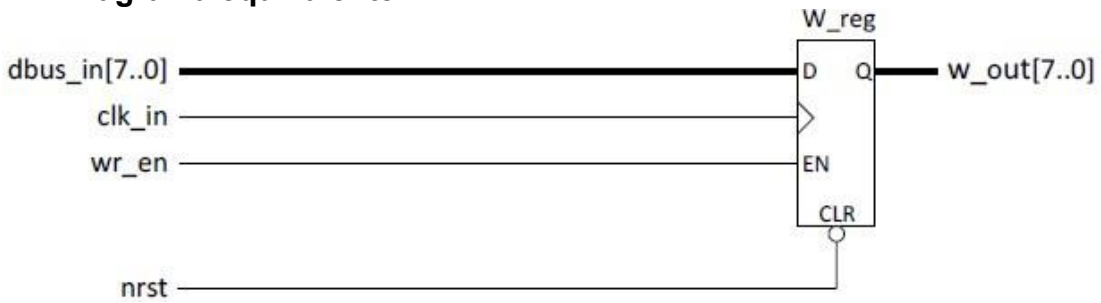

Figura 1: Diagrama equivalente do W_REG.

II.III - Descrição W_reg em VHDL

Primeiro houve a declaração das bibliotecas e das entidades que vamos utilizar, o q_out se refere ao w_out.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity W_reg is -- Entidade para W_reg
    Port (
        clk_in : in STD_LOGIC; -- Entrada do clock
        nrst : in STD_LOGIC; -- Entrada de reset assíncrono
        d_in : in STD_LOGIC_VECTOR (7 downto 0); -- Vetor de entrada de 8 bits para o registrador
        wr_en : in STD_LOGIC; -- Entrada de habilitação para escrita no registrador
        q_out : out STD_LOGIC_VECTOR (7 downto 0) -- Vetor de saída de 8 bits do registrador
    );
end W_reg;
```

Figura 2: Bibliotecas e Entidade W_reg.

Após a declaração das entidades definimos a arquitetura

```
architecture register_8bits of W_reg is -- Arquitetura para um registrador de 8 bits
    signal q_temp : STD_LOGIC_VECTOR (7 downto 0) := (others => '0'); -- Vetor temporário para atribuição de dados de saída
begin
    process (clk_in, nrst)
    begin
        if rising_edge(clk_in) then -- Se o sinal do clock está em subida
            if nrst = '0' then -- Se o bit de reset está ativo, definir a saída como 0
                q_temp <= "00000000";
            elsif wr_en = '1' then -- Se o controle de escrita estiver em 1, atualizar o registrador com os dados de entrada
                q_temp <= d_in;
            end if;
        end if;
    end process;

    q_out <= q_temp;
end register_8bits;
```

Figura 3: Arquitetura W_REG.

II.IV - Simulação e teste no Quartus II

Para conduzir a simulação, configuramos um cenário abrangente que engloba todas as operações afetadas pelos sinais de entrada. O sinal de clock opera com um período de 10ns. Durante o início da simulação, ativamos o sinal nRST para avaliar a prioridade dessa entrada em relação às demais. Após sua desativação, quando a entrada é habilitada, o valor de D_IN é instantaneamente refletido na saída na borda de subida do sinal de clock.

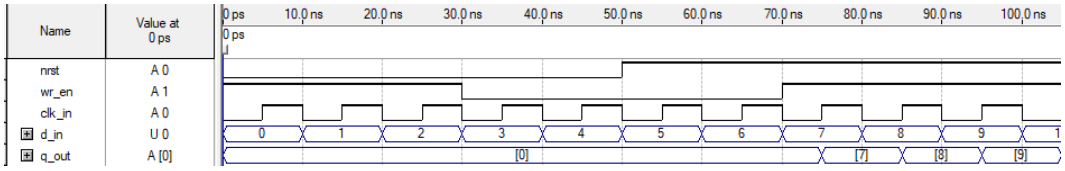

Figura 4: Funcionamento W_REG.

III. FSR_REG

O registrador FSR_REG é constituído por um registrador de 8 bits, equipado com 6 entradas e duas saídas de 8 bits. Ele inclui uma entrada nRST, ativada em nível lógico baixo, cuja ativação resulta na reinicialização de todos os bits das saídas, priorizando-a sobre as demais entradas. Em seguida, temos o sinal CLK_IN, uma entrada de clock que habilita as operações de escrita no registrador. A terceira entrada, ABUS_IN, funciona como entrada de endereçamento, determinando quando o registrador deve ser lido ou escrito, com o endereço correspondendo a 'XX0000100'.

Na sequência, encontramos DBUS_IN, uma entrada de 8 bits, que representa o valor a ser escrito nas saídas, caso a entrada WR_EN esteja ativa. Por fim, temos as entradas WR_EN, que habilita a operação de escrita no registrador na borda de subida do sinal de clock, e RD_EN, que habilita a operação de leitura, desde que o endereço presente em ABUS_IN seja o correto.

III.I - Entradas e Saídas

Entradas:

- nRST: Reset assíncrono, zera o registrador quando ativado (nível lógico baixo), com prioridade sobre outras entradas.
- CLK_IN: Sinal de clock para escrita no registrador, operando na borda de subida do clock, se habilitado.
- ABUS_IN[8..0]: Entrada de endereçamento; o registrador é escrito ou lido quando ABUS_IN[6..0] é igual a "0000100."
- DBUS_IN[7..0]: Entrada de dados para escrita no registrador, habilitada por WR_EN, com endereçamento por ABUS_IN.
- WR_EN: Entrada de habilitação para escrita no registrador; ativa (nível lógico alto) permite escrita sincronizada com CLK_IN, se o endereço especificado estiver presente em ABUS_IN.
- RD_EN: Entrada de habilitação para leitura; ativa (nível lógico alto) faz DBUS_OUT corresponder ao conteúdo do registrador, se o endereço especificado estiver presente em ABUS_IN.

Saídas:

- DBUS_OUT[7..0]: Saída de dados lidos, habilitada por RD_EN e endereçada por ABUS_IN. Em repouso, fica em alta impedância ("ZZZZZZZZ").
- FSR_OUT[7..0]: Saída correspondente ao conteúdo do registrador, sempre ativa, independentemente de habilitações.

III.II - Diagrama equivalente

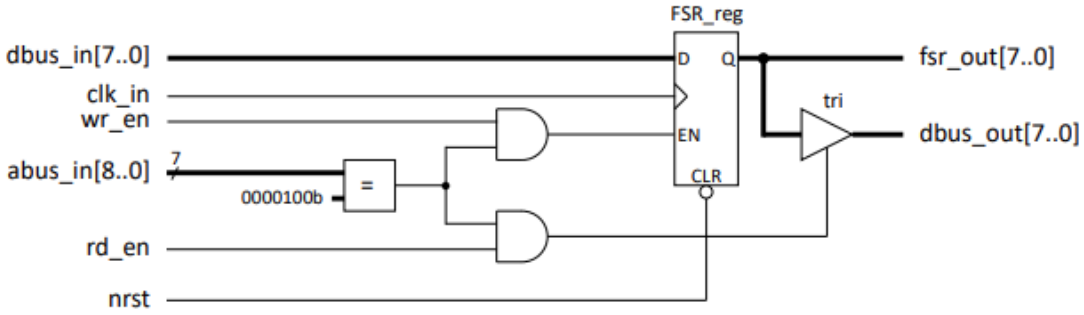

Figura 5: Diagrama equivalente do FSR_REG.

III.III - Descrição FSR_REG em VHDL

Primeiro houve a declaração das bibliotecas e das entidades que vamos utilizar.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity FSR_reg is
    Port (
        nrst : in STD_LOGIC;          -- Entrada de reset assíncrono
        clk_in : in STD_LOGIC;        -- Entrada de clock para escrita no registrador
        abus_in : in STD_LOGIC_VECTOR(8 downto 0);  -- Entrada de endereçamento
        dbus_in : in STD_LOGIC_VECTOR(7 downto 0);  -- Entrada de dados para escrita no registrador
        wr_en : in STD_LOGIC;          -- Entrada de habilitação para escrita no registrador
        rd_en : in STD_LOGIC;          -- Entrada de habilitação para leitura
        dbus_out : out STD_LOGIC_VECTOR(7 downto 0);  -- Saída de dados lidos
        fsr_out : out STD_LOGIC_VECTOR(7 downto 0)  -- Saída correspondente ao registrador
    );
end FSR_reg;
```

Figura 6: Entidade FSR_REG.

Após a declaração das entidades definimos a arquitetura.

```
architecture architeture_8bits of FSR_reg is
    signal temp : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');
begin
    process (clk_in, nrst)
    begin
        if nrst = '0' then
            -- Reset assíncrono ativo, zera o registrador
            temp <= (others => '0');
        elsif rising_edge(clk_in) then
            -- Verifica se o endereço é "0000100" (7 bits menos significativos)
            if abus_in(8 downto 2) = "0000100" then
                if wr_en = '1' then
                    -- Escreve no registrador sincronamente com o clock
                    temp <= dbus_in;
                end if;
                if rd_en = '1' then
                    -- Leitura combinacional, apenas habilitada
                    dbus_out <= temp;
                else
                    -- Leitura desabilitada, saída fica em alta impedância
                    dbus_out <= (others => 'Z');
                end if;
            end if;
        end if;
    end process;

    -- A saída fsr_out está sempre ativa e corresponde ao conteúdo do registrador
    fsr_out <= temp;
end architeture_8bits;
```

Figura 7: Arquitetura FSR_REG.

III.IV - Simulação e teste no Quartus II

Durante a simulação, preparamos um cenário que abrange todas as operações influenciadas pelos sinais de entrada, com um clock operando a um período de 10ns. Inicialmente, ativamos o sinal nRST no início da simulação para testar a prioridade dessa entrada em relação às demais.

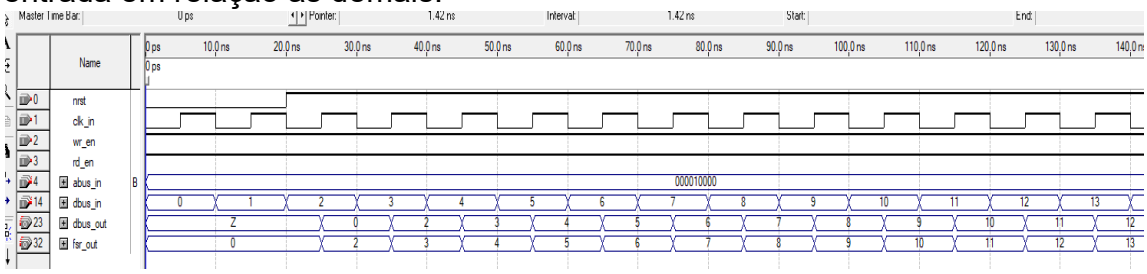

Figura 8: Funcionamento FSR_REG.

IV. STATUS_REG

O STATUS_REG é composto por um registrador de 8 bits, acompanhado por 12 entradas e seis saídas. Inclui uma entrada nRST, ativada em nível lógico baixo, que, quando acionada, zera todos os bits das saídas, priorizando-se sobre as demais entradas. Em seguida, encontramos o sinal CLK_IN, uma entrada de clock que permite habilitar a operação de escrita no registrador.

A terceira entrada, ABUS_IN, atua como entrada de endereçamento, determinando que o registrador deve ser escrito ou lido quando ABUS_IN[7..0] for igual a "XX0000011". Na sequência, temos DBUS_IN, uma entrada de 8 bits, que representa o valor a ser gravado nas saídas, caso WR_EN esteja ativo.

Além disso, existem três entradas adicionais, Z_WR_EN, DC_WR_EN e C_WR_EN, cada uma com um bit, que servem para alterar os três bits menos significativos do registrador. A entrada Z_IN afeta o segundo bit, a DC_IN o primeiro bit e a C_IN o bit 0, mas essas alterações só ocorrem quando as respectivas entradas de habilitação estiverem ativas. A entrada RD_EN habilita a leitura, desde que o endereço especificado em ABUS_IN seja correto.

IV.I - Entradas e Saídas

Entradas:

- nRST: Reset assíncrono que zera todos os bits do registrador quando ativado (nível lógico baixo), com prioridade.
- CLK_IN: Sinal de clock para escrita no registrador; escrita ocorre na borda de subida do clock, se habilitada.
- ABUS_IN[8..0]: Entrada de endereçamento; o registrador é escrito ou lido quando ABUS_IN[6..0] é igual a "0000011".
- DBUS_IN[7..0]: Entrada de dados para escrita no registrador, habilitada por WR_EN e endereçada por ABUS_IN.
- WR_EN: Entrada de habilitação para escrita no registrador; ativa (nível lógico alto) permite escrita sincronizada com CLK_IN, se o endereço especificado estiver presente em ABUS_IN.
- RD_EN: Entrada de habilitação para leitura; ativa (nível lógico alto) faz DBUS_OUT corresponder ao conteúdo do registrador, exceto pelos bits 4 e 3, que devem ser sempre '1'.
- Z_IN, DC_IN, C_IN: Entradas de dados para escrita nos bits 2, 1 e 0 do registrador, respectivamente, habilitadas por Z_WR_EN, DC_WR_EN, C_WR_EN.
- Z_WR_EN, DC_WR_EN, C_WR_EN: Entradas de habilitação para escrita nos bits 2, 1 e 0 do registrador, respectivamente. Têm prioridade sobre WR_EN.

Saídas:

- DBUS_OUT[7..0]: Saída de dados lidos, habilitada por RD_EN e endereçada por ABUS_IN. Em repouso, fica em alta impedância ("ZZZZZZZZ").
- IRP_OUT: Saída correspondente ao bit 7 do registrador, sempre ativa, independente de habilitações.

- RP_OUT[1..0]: Saída correspondente aos bits 6 e 5 do registrador, sempre ativa, independente de habilitações.
- Z_OUT: Saída correspondente ao bit 2 do registrador, sempre ativa, independente de habilitações.
- DC_OUT: Saída correspondente ao bit 1 do registrador, sempre ativa, independente de habilitações.
- C_OUT: Saída correspondente ao bit 0 do registrador, sempre ativa, independente de habilitações.

IV.II - Diagrama equivalente

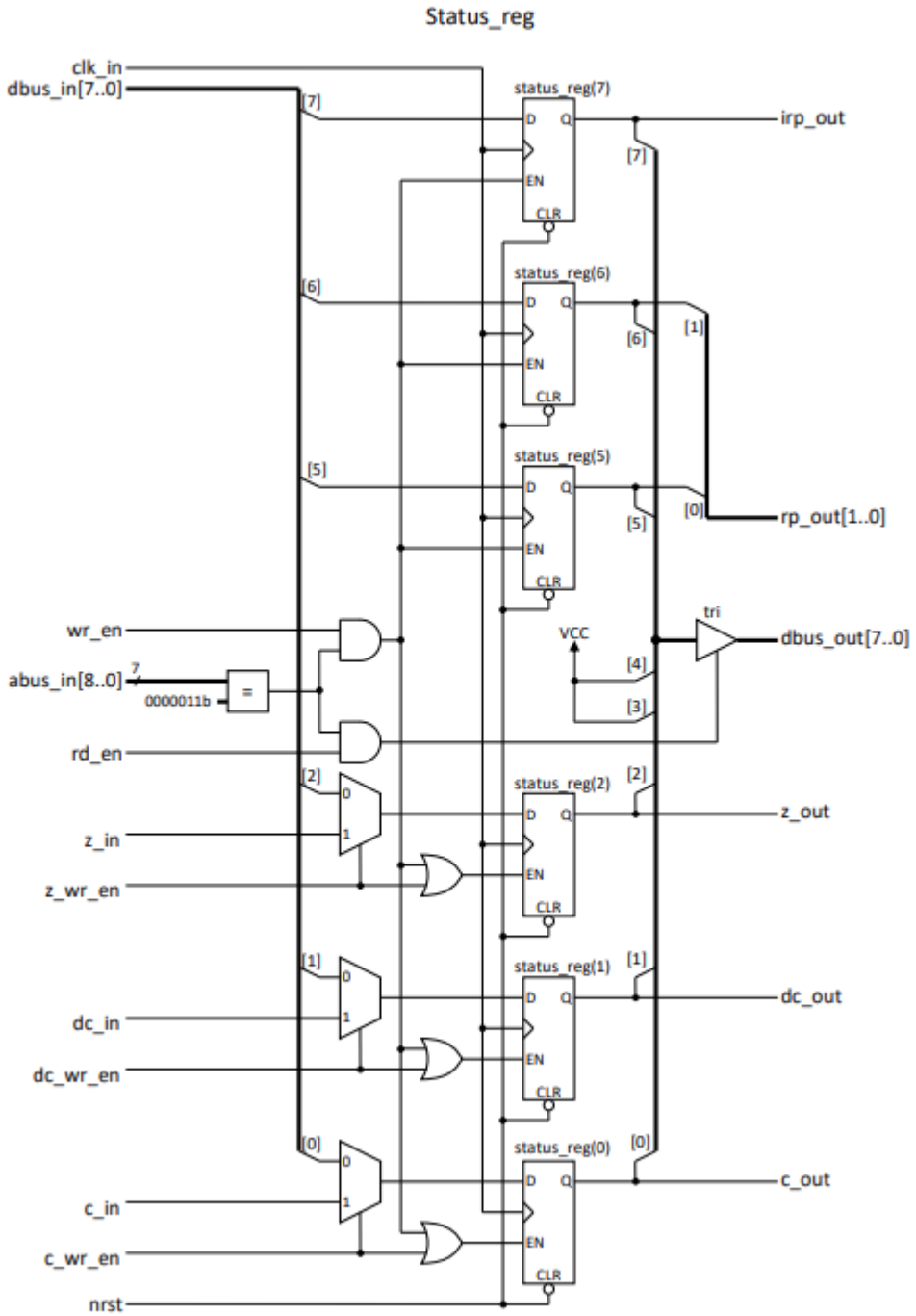

Figura 9: Diagrama equivalente do STATUS_REG.

IV.III - Descrição STATUS_REG em VHDL

Primeiro houve a declaração das bibliotecas e das entidades que vamos utilizar.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity Status_reg is
    Port (
        nrst : in STD_LOGIC;          -- Entrada de reset assíncrono
        clk_in : in STD_LOGIC;        -- Entrada de clock para escrita no registrador
        abus_in : in STD_LOGIC_VECTOR(8 downto 0);  -- Entrada de endereçamento
        dbus_in : in STD_LOGIC_VECTOR(7 downto 0);  -- Entrada de dados para escrita no registrador
        wr_en : in STD_LOGIC;          -- Entrada de habilitação para escrita no registrador
        rd_en : in STD_LOGIC;          -- Entrada de habilitação para leitura
        z_in : in STD_LOGIC;           -- Entrada de dado para escrita no bit 2 do registrador
        dc_in : in STD_LOGIC;          -- Entrada de dado para escrita no bit 1 do registrador
        c_in : in STD_LOGIC;           -- Entrada de dado para escrita no bit 0 do registrador
        z_wr_en : in STD_LOGIC;        -- Entrada para habilitação da escrita no bit 2 do registrador
        dc_wr_en : in STD_LOGIC;       -- Entrada para habilitação da escrita no bit 1 do registrador
        c_wr_en : in STD_LOGIC;        -- Entrada para habilitação da escrita no bit 0 do registrador
        dbus_out : buffer STD_LOGIC_VECTOR(7 downto 0);  -- Saída de dados lidos
        wr_out : out STD_LOGIC_VECTOR(7 downto 0); -- saida correspondente ao registrador - adicionei não pedia no trabalho -
        irp_out : out STD_LOGIC;       -- Saída correspondente ao bit 7 do registrador
        rp_out : out STD_LOGIC_VECTOR(1 downto 0);    -- Saída correspondente aos bits 6 e 5 do registrador
        z_out : out STD_LOGIC;         -- Saída correspondente ao bit 2 do registrador
        dc_out : out STD_LOGIC;        -- Saída correspondente ao bit 1 do registrador
        c_out : out STD_LOGIC           -- Saída correspondente ao bit 0 do registrador
    );
end Status_reg;
```

Figura 10: Bibliotecas e Entidade STATUS_REG.

Após a declaração das entidades definimos a arquitetura

```vhdl
architecture register_8bits of Status_reg is

    signal temp : STD_LOGIC;
begin
    process (clk_in, nrst)
    begin
    temp <= '1';
        if nrst = '0' then
            -- Reset assíncrono ativo, zera o registrador
            dbus_out <= (others => '0');
        elsif rising_edge(clk_in) then
            -- Verifica se o endereço é "0000011" (7 bits menos significativos)
            if abus_in(8 downto 2) = "0000011" then -- se o endereço de abus_in for o correto, permite as operações definidas abaixo:

                if wr_en = '1' then -- permite a escrita no registrador
                    wr_out <= dbus_in;  -- saida correspondente a escrita do registrador
                end if;
                if rd_en = '1' then -- a saida dbus_out deverá corresponder ao conteúdo do registrador, exceto os bits 4 e 3, que deverão ser sempre lidos como '1'.
                    dbus_out <= dbus_in;
                    dbus_out(4) <= temp; -- os bits 4 e 3 do registrador devem ser sempre lidos como 1
                    dbus_out(3) <= temp;
                else
                    dbus_out <= (others => 'Z'); -- se rd_en for diferente de 1 a saida permanece em alta impedancia
                end if;
                -- as operações a seguir tem preferencia sobre rd_in
                if z_wr_en = '1' then -- quando habilitada permite a escrita no bit 2 do registrador
                    dbus_out(2) <= z_in; -- bit 2 do registrador passa a ser o valor de z_in
                end if;
                if dc_wr_en = '1' then -- quando habilitada permite a escrita no bit 1 do registrador
                    dbus_out(1) <= dc_in; -- bit 1 do registrador passa a ser o valor de dc_in
                end if;
                if c_wr_en = '1' then -- quando habilitada permite a escrita no bit 0 do registrador
                    dbus_out(0) <= c_in; -- bit 0 do registrador passa a ser o valor de c_in
                end if;
            else -- caso o endereço de abus_in não seja correto a saida fica em alta impedancia
                dbus_out <= (others => 'Z');
                wr_out <= (others => 'Z');
            end if;
        end if;
    end process;
```

Figura 11: Arquitetura STATUS_REG.

```vhdl
    -- As saídas irp_out, rp_out, z_out, dc_out, c_out correspondem aos bits específicos do registrador

    irp_out <= dbus_out(7);
    rp_out <= dbus_out(6 downto 5);
    z_out <= dbus_out(2);
    dc_out <= dbus_out(1);
    c_out <= dbus_out(0);
end register_8bits;
```

Figura 12: Saídas arquitetura STATUS_REG.

IV.IV - Simulação e teste no Quartus II

Para conduzir a simulação, estabelecemos um cenário que abrange todas as operações decorrentes dos sinais de entrada, com o clock operando a um período de 2ns.

Durante o início da simulação, ativamos o sinal nRST para avaliar sua prioridade em relação às demais entradas.

Após a desativação do reset, mesmo com a entrada habilitada, os valores da saída não são alterados devido ao ABUS_IN estar com um valor incorreto. No entanto, ao ajustá-lo, a saída passa a refletir o valor da entrada DBUS_IN.

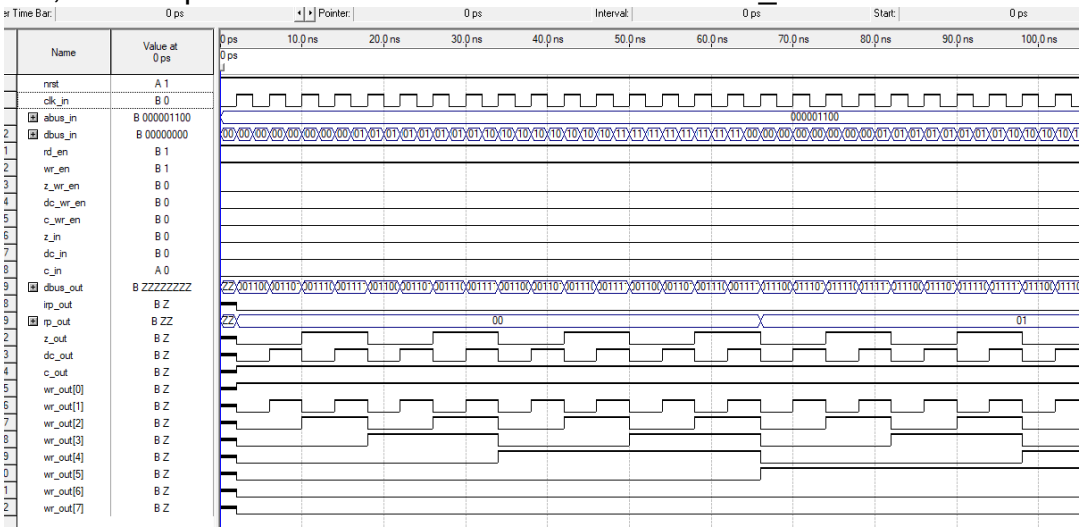

Figura 13: Funcionamento 1 STATUS_REG.

Posteriormente, conduzimos os testes das seis últimas entradas, configurando o valor de cada bit para 0 ao habilitar as respectivas alterações.

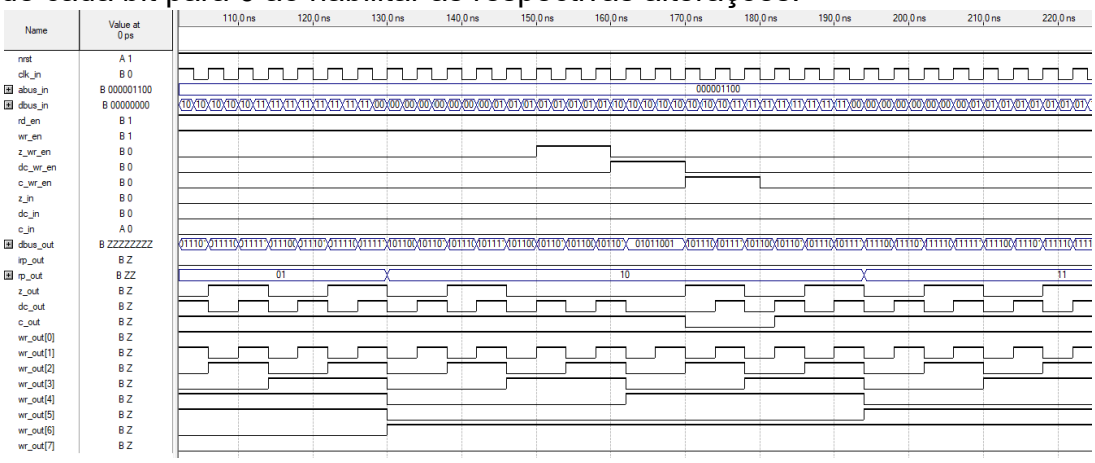

Figura 14: Funcionamento 2 STATUS_REG.

V. STACK

O STACK é composto por um conjunto de 8 registradores de 13 bits, com 5 entradas e uma saída. Inclui uma entrada nRST, ativada em nível lógico baixo, que, quando acionada, zera todos os bits dos registradores e tem prioridade sobre outras entradas. Em seguida, encontramos o sinal CLK_IN, uma entrada de clock que habilita a escrita nos registradores.

A terceira entrada, STACK_IN, é o valor a ser armazenado na pilha, no primeiro registrador. Posteriormente, temos a entrada STACK_PUSH, que, quando ativada, escreve o valor presente em STACK_IN no primeiro registrador da pilha. Simultaneamente, o segundo registrador recebe o conteúdo do primeiro, e assim por diante, até o oitavo registrador, que recebe o conteúdo do sétimo registrador.

A última entrada, STACK_POP, quando ativada, transfere o conteúdo do segundo registrador para o primeiro, o conteúdo do terceiro registrador para o segundo, e assim por diante, até o sétimo registrador, que recebe o conteúdo do oitavo. O oitavo registrador, por sua vez, recebe o valor zero, com todos os bits igual a ‘0’.

V.I - Entradas e Saídas

Entradas:

- nRST: Reset assíncrono que zera todos os bits dos registradores quando ativado (nível lógico baixo), com prioridade.
- CLK_IN: Sinal de clock para escrita nos registradores; a escrita ocorre na borda de subida do clock, se habilitada.
- STACK_IN[12..0]: Entrada de dados para a pilha.
- STACK_PUSH: Entrada de habilitação para empilhar valores. Quando ativada (nível lógico alto), o valor em STACK_IN é escrito no primeiro registrador da pilha, e os valores são deslocados para cima na pilha.
- STACK_POP: Entrada de habilitação para desempilhar valores. Quando ativada (nível lógico alto), os valores na pilha são deslocados para baixo, e o valor zero é colocado no último registrador.

Saídas:

- STACK_OUT[12..0]: Saída correspondente ao topo da pilha, sempre ativa e não dependente de habilitações.

V.II - Diagrama equivalente

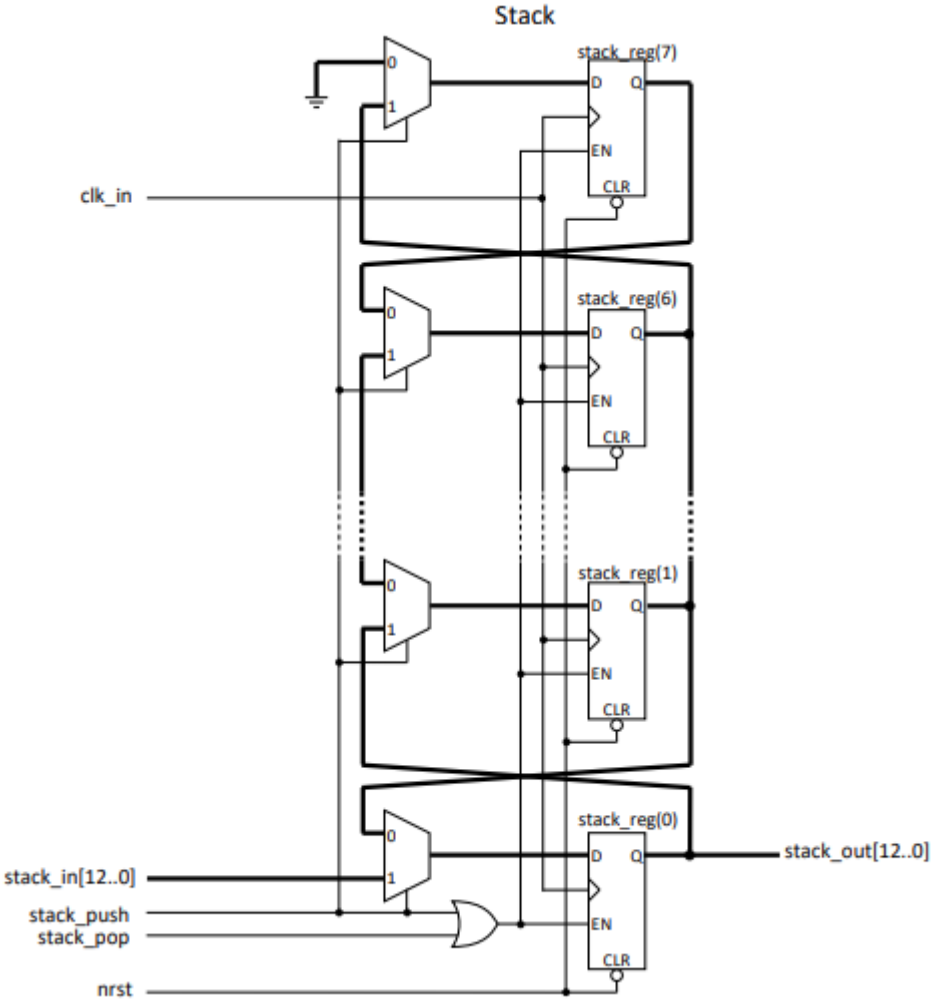

Figura 15: Diagrama equivalente do STACK.

V.III - Descrição STACK em VHDL

Primeiro houve a declaração das bibliotecas e das entidades que vamos utilizar.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity Stack is
    Port ( clk_in : in STD_LOGIC; -- entrade de clock
           nrst : in STD_LOGIC; -- entrada de reset
           stack_in : in STD_LOGIC_VECTOR (12 downto 0); --vetor de entrada de dados da pilha
           stack_push : in STD_LOGIC;
           stack_pop : in STD_LOGIC;
           stack_out : out STD_LOGIC_VECTOR (12 downto 0));
end Stack;
```

Figura 16: Bibliotecas e Entidade STACK.

Após a declaração das entidades definimos a arquitetura

```
architecture architectureStack of Stack is
    type RegStack is array (0 to 7) of STD_LOGIC_VECTOR (12 downto 0);
    signal Regs : RegStack;
begin

process (clk_in, nrst)
begin
    if nrst = '0' then
        for i in 0 to 7 loop
            Regs(i) <= (others => '0');
        end loop;
    elsif rising_edge(clk_in) then
        if stack_push = '1' then
            Regs(0) <= stack_in;
            for i in 1 to 7 loop
                Regs(i) <= Regs(i-1);
            end loop;
        elsif stack_pop = '1' then
            Regs(7) <= (others => '0');
            for i in 6 downto 0 loop
                Regs(i+1) <= Regs(i);
            end loop;
        end if;
    end if;
end process;

stack_out <= Regs(7);

end architectureStack;
```

Figura 17: Arquitetura STACK.

V.IV - Simulação e teste no Quartus II

Para conduzirmos a simulação, configuramos um cenário abrangendo todas as operações influenciadas pelos sinais de entrada, com o clock operando a um período de 10ns.

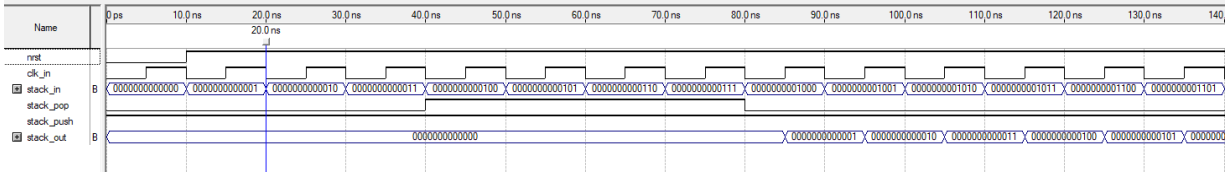

Figura 18: Funcionamento STACK.

VI. PC_REG

O bloco PC_reg é uma parte crucial do sistema, compreendendo um registrador de 13 bits denominado PC (Program Counter) e outro registrador de 8 bits conhecido como PCLATH. Este componente desempenha um papel fundamental na gestão do fluxo de instruções do sistema, permitindo o rastreamento da próxima instrução a ser executada. Além disso, o PC_reg interage de forma integrada com a pilha (stack) desenvolvida no item 5 do sistema. É importante observar que a parte de baixa ordem do registrador PC, representada pelos bits 7 a 0, é comumente denominada de PCL

VI.I - Entradas e Saídas

Entradas:

- nRST: Reset assíncrono que zera os registradores PC e PCLATH quando ativado (nível lógico baixo), com prioridade.
- CLK_IN: Sinal de clock para incremento e carga dos registradores; a ação ocorre na borda de subida do clock, se habilitada.
- ADDR_IN[10..0]: Entrada de dados para carga no registrador PC, habilitada por LOAD_EN.
- ABUS_IN[8..0]: Entrada de endereçamento para PCL e PCLATH. PCL é escrito ou lido quando ABUS_IN[6..0] = “0000010”, e PCLATH é escrito ou lido quando ABUS_IN[6..0] = “0001010”.
- DBUS_IN[7..0]: Entrada de dados para escrita em PCL e PCLATH, habilitada por WR_EN e endereçada por ABUS_IN.
- INC_PC: Entrada de habilitação para incremento do registrador PC.
- LOAD_PC: Entrada de habilitação para carga do PC.
- WR_EN: Entrada de habilitação para escrita nos registradores.
- RD_EN: Entrada de habilitação para leitura.
- STACK_PUSH: Entrada de habilitação para empilhar valores na pilha.
- STACK_POP: Entrada de habilitação para desempilhar valores da pilha.

Saídas:

- NEXTPC_OUT[12..0]: Saída do valor a ser carregado no contador (registrador PC); sempre ativa, não depende de habilitação.
- DBUS_OUT[7..0]: Saída de dados lidos, habilitada por RD_EN e endereçada por ABUS_IN. Em repouso, fica em alta impedância ("ZZZZZZZZ").

VI.II - Diagrama equivalente

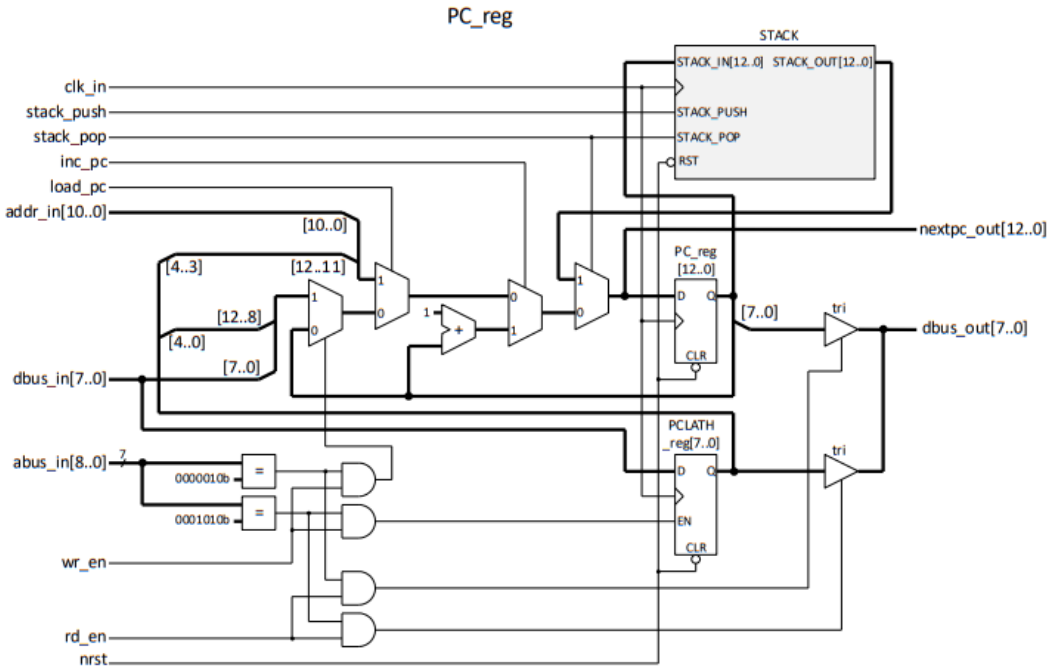

Figura 19: Diagrama equivalente do PC_REG.

VI.III - Descrição W_reg em VHDL

Primeiro houve a declaração das bibliotecas e das entidades que vamos utilizar, o q_out se refere ao w_out.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity PC_reg is
    Port (
        clk_in: IN STD_LOGIC;
        nrst: IN STD_LOGIC;
        addr_in: IN STD_LOGIC_VECTOR(10 DOWNTO 0);
        abus_in: IN STD_LOGIC_VECTOR(8 DOWNTO 0);
        dbus_in: IN STD_LOGIC_VECTOR(7 DOWNTO 0);
        inc_pc: IN STD_LOGIC;
        load_pc: IN STD_LOGIC;
        wr_en: IN STD_LOGIC;
        rd_en: IN STD_LOGIC;
        stack_push: IN STD_LOGIC;
        stack_pop: IN STD_LOGIC;
        nextpc_out: OUT STD_LOGIC_VECTOR(12 DOWNTO 0);
        dbus_out: OUT STD_LOGIC_VECTOR(7 DOWNTO 0)
        );
end entity;
```

Figura 20: Bibliotecas e Entidade PC_REG.

Após a declaração das entidades definimos a arquitetura

```
ARCHITECTURE arch1 OF PC_reg IS
BEGIN
        PROCESS (nrst, clk_in, addr_in, abus_in, dbus_in, inc_pc, load_pc, wr_en, rd_en, stack_push, stack_pop)
            VARIABLE regPC: STD_LOGIC_VECTOR(12 DOWNTO 0);
            VARIABLE regPCLATH: STD_LOGIC_VECTOR(7 DOWNTO 0);
            VARIABLE reg0: STD_LOGIC_VECTOR(12 DOWNTO 0);
            VARIABLE reg1: STD_LOGIC_VECTOR(12 DOWNTO 0);
            VARIABLE reg2: STD_LOGIC_VECTOR(12 DOWNTO 0);
            VARIABLE reg3: STD_LOGIC_VECTOR(12 DOWNTO 0);
            VARIABLE reg4: STD_LOGIC_VECTOR(12 DOWNTO 0);
            VARIABLE reg5: STD_LOGIC_VECTOR(12 DOWNTO 0);
            VARIABLE reg6: STD_LOGIC_VECTOR(12 DOWNTO 0);
            VARIABLE reg7: STD_LOGIC_VECTOR(12 DOWNTO 0);
        BEGIN
            IF nrst = '0' THEN
                regPC := "0000000000000";
                regPCLATH := "00000000";
                reg0 := "0000000000000";
                reg1 := "0000000000000";
                reg2 := "0000000000000";
                reg3 := "0000000000000";
                reg4 := "0000000000000";
                reg5 := "0000000000000";
                reg6 := "0000000000000";
                reg7 := "0000000000000";
        ELSE
            IF stack_pop = '1' AND RISING_EDGE(clk_in) THEN
                reg0 := reg1;
                reg1 := reg2;
                reg2 := reg3;
                reg3 := reg4;
                reg4 := reg5;
                reg5 := reg6;
                reg6 := reg7;
                reg7 := "0000000000000";
```

Figura 21: Arquitetura PC_REG.

```
            ELSIF inc_pc = '1' AND RISING_EDGE(clk_in) THEN regPC := regPC + "0000000000001";
            ELSIF load_pc = '1' THEN regPC (10 DOWNTO 0) := addr_in;
                regPC (12 DOWNTO 11) := regPCLATH (4 DOWNTO 3);
            ELSIF wr_en = '1' AND RISING_EDGE(clk_in) THEN IF abus_in(6 DOWNTO 0) = "0000010" THEN regPC(7 DOWNTO 0) := dbus_in;
                regPC(12 DOWNTO 8) := regPCLATH (4 DOWNTO 0);
            ELSIF abus_in(6 DOWNTO 0) = "0001010" THEN regPCLATH := dbus_in;
            END IF;
            END IF;
            IF stack_push = '1' AND RISING_EDGE (clk_in) THEN
                reg7 := reg6;
                reg6 := reg5;
                reg5 := reg4;
                reg4 := reg3;
                reg3 := reg2;
                reg2 := reg1;
                reg1 := reg0;
                reg0 := regPC;
            END IF;
        END IF;
        IF rd_en = '1' AND abus_in(6 DOWNTO 0) = "0000010" THEN dbus_out <= regPC(7 DOWNTO 0);
        ELSIF rd_en = '1' AND abus_in (6 DOWNTO 0) = "0001010" THEN dbus_out <= regPCLATH;
        ELSE dbus_out <= "ZZZZZZZZ";
        END IF;
    end process;
end arch1;
```

Figura 22: Arquitetura 2 PC_REG.

VI.IV - Simulação e teste no Quartus II

Para a realização da simulação, configuramos um cenário abrangendo todas as operações decorrentes dos sinais de entrada, com o clock operando a um período de 5ns. Inicialmente, ativamos o sinal nRST no início da simulação para avaliar sua prioridade em relação às demais entradas. Após a desativação do reset, adicionamos o valor de INC_EN, o que resultou em um aumento sequencial da saída de um em um a cada borda de subida do clock.

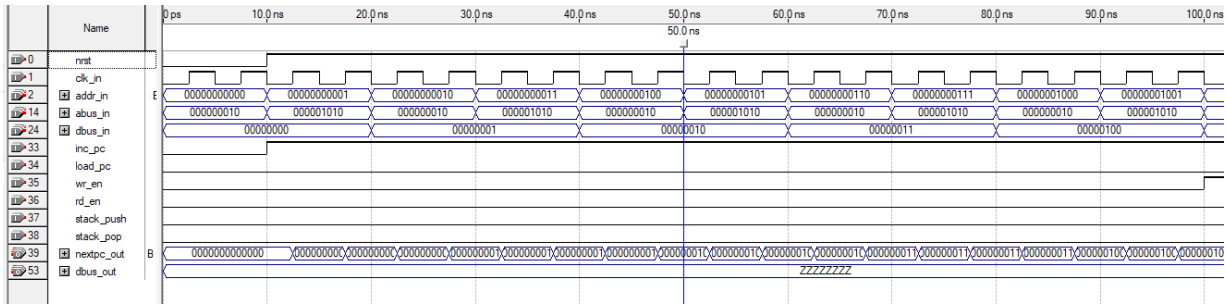

Figura 23: Funcionamento 1 PC_REG.

Em seguida, procedemos ao acionamento de LOAD_EN, o que resultou no carregamento do valor ADDR_IN na saída.

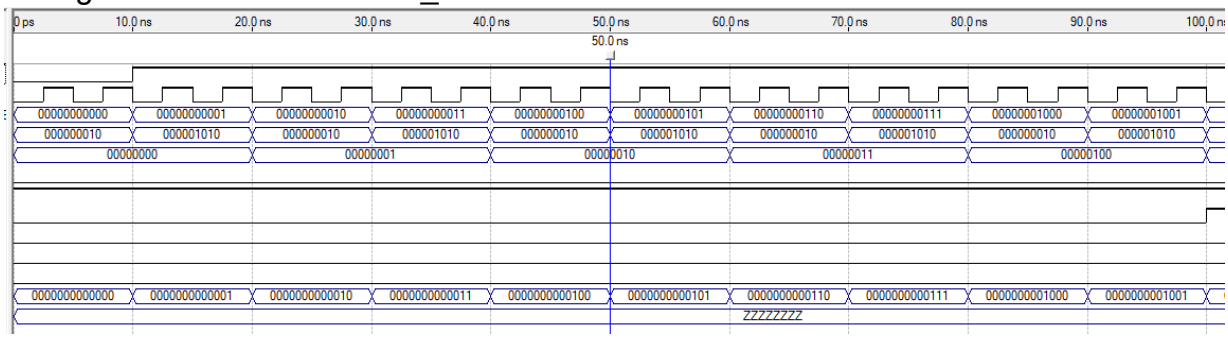

Figura 24: Funcionamento 2 PC_REG.

Avançamos para a etapa seguinte, na qual ativamos WR_EN, possibilitando o carregamento do valor DBUS_IN na saída. Nesse processo, os bits mais significativos foram modificados de acordo com o valor presente em ABUS_IN.

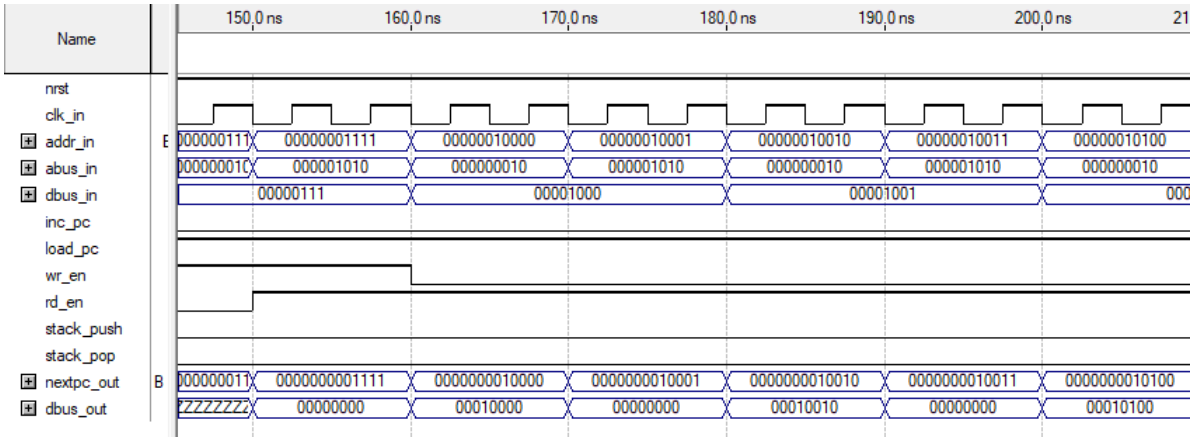

Figura 25: Funcionamento 3 PC_REG.

VII. CONCLUSÃO

Ao concluir esta etapa, estabelecemos uma base sólida para a compreensão e implementação eficaz dos registradores essenciais em nosso projeto. Esses componentes desempenham um papel central na execução de instruções e no gerenciamento de dados, proporcionando uma estrutura robusta para o funcionamento do sistema como um todo. Essa abordagem detalhada e as simulações realizadas nos permitem prosseguir com confiança na próxima fase do projeto, sabendo que os registradores essenciais estão funcionando de acordo com nossos requisitos.

VIII. REFERENCIAS

Material disponível na disciplina de Sistemas Reconfiguráveis. Canvas Puc Minas, 2023. Disponível em: https://pucminas.instructure.com/courses/161258. Acesso em: 12 out. 2023.

Tutorials Point. Binary Arithmetic, 2023. Disponível em: https://www.tutorialspoint.com/computer_logical_organization/binary_arithmetic.htm. Acesso em: 12 out. 2023.